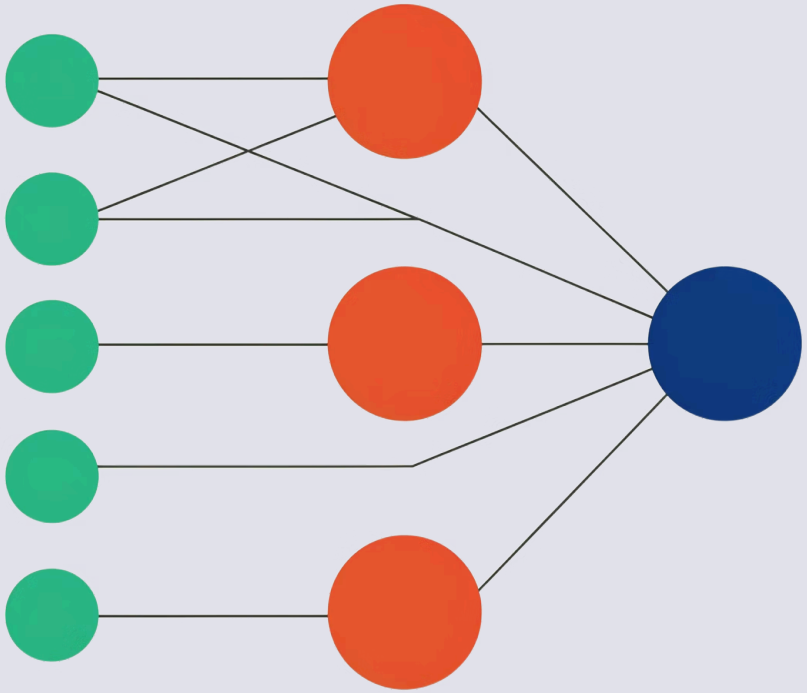


Types of LLM Models

Choosing the right model for the job

Introduction to Generative AI



Not All LLMs Are Built the Same

LLMs are deep learning systems trained on massive text corpora to understand and generate language

They differ in architecture, training strategy, size, and intended use

Before choosing a model for a real application, you need to know these differences



Case Study: TechCorp's Software Assistant

TechCorp wants an internal AI assistant that can:

- **Answer questions about their codebase**
- **Summarise meeting transcripts**
- **Generate boilerplate code**
- **Classify customer support tickets**

Which type of LLM fits each task? Let's follow their engineering team.

The Six Categories We'll Cover

01

Base models

02

Instruction-tuned models

03

RLHF models

04

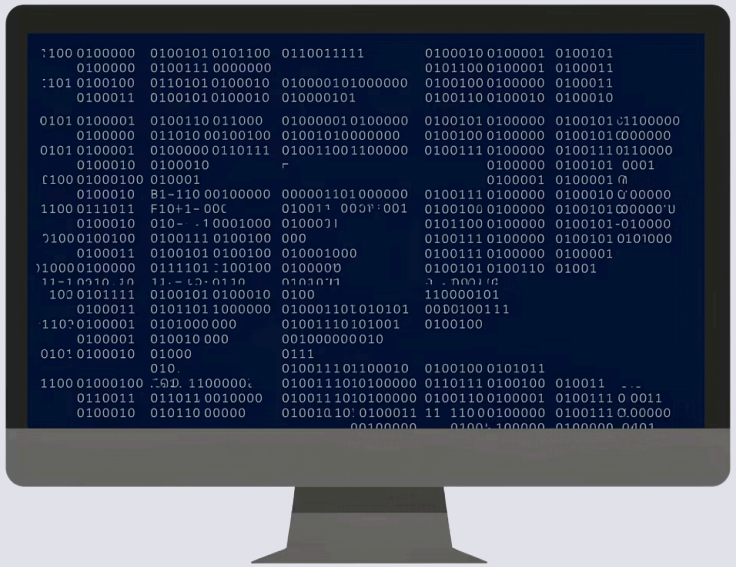
Domain-specific fine-tuned models

05

Multimodal models

06

Small Language Models (SLMs)



1. Base Models (Pretrained)

Trained on a large corpus of raw text

Objective: next-token prediction (self-supervised)

Learns grammar, facts, reasoning patterns, and code structure purely from exposure to text

Examples: GPT-2, LLaMA 2 Base, Mistral 7B Base

Base Models: How They Work

Training objective: predict the next token given the previous tokens.

Input

"The capital of France is"

Output

"Paris"

The model learns from billions of such sequences, **with no human labelling**.

Base Models: TechCorp's Experience

TechCorp asks LLaMA 2 Base a developer question:

Prompt

"What does the function processPayment() do?"

Output

"What does the function processPayment() do? It is defined in the file payment_handler.py and is called from checkout.py..."

The model **continues the text** instead of answering. It was never taught to follow instructions.

TechCorp realises they need something more.

Base Models: When to Use



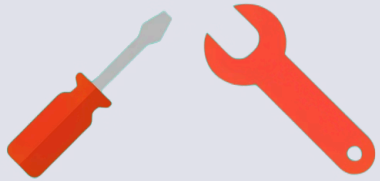
Research and experimentation

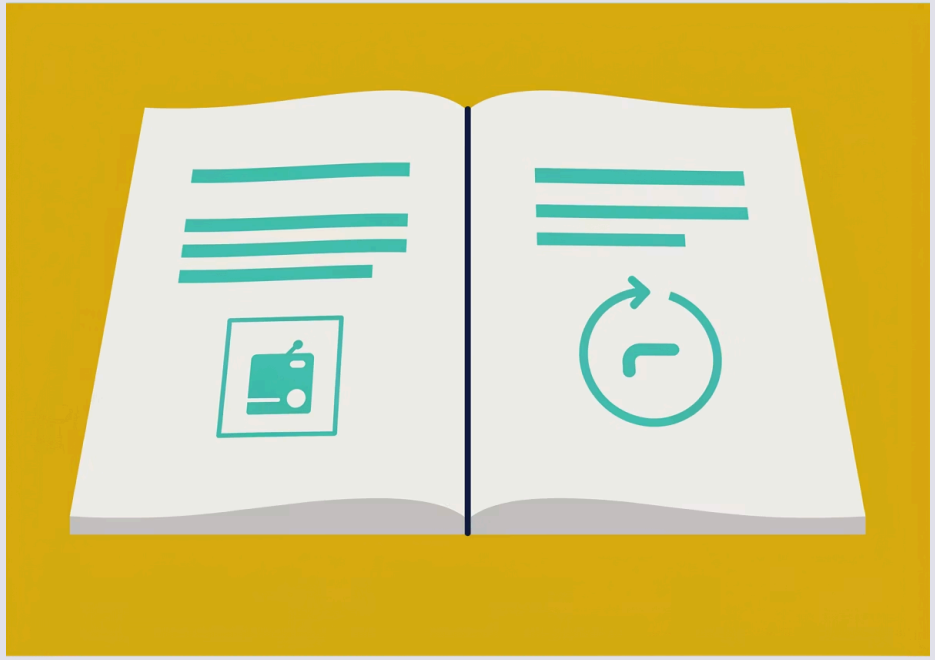


A starting point for fine-tuning



Text-completion pipelines





2. Instruction-Tuned Models

A base model trained further on curated (instruction, response) pairs

Learns to follow directions from a user

Learns the expected format and tone of helpful responses

Examples: GPT-3.5-turbo, LLaMA 2 Chat, Mistral Instruct, Flan-T5

Instruction Tuning: How It Works

Fine-tuning data format:

```
[INST] Summarize the following meeting notes: ... [/INST] The team agreed to release version 2.0 by Friday...
```

The model sees thousands of examples like this and learns what a good answer looks like.

Instruction-Tuned: TechCorp's Experience

TechCorp switches to Mistral 7B Instruct. Same question, new result:

Prompt

"What does the function processPayment() do?"

Output

"The function processPayment() handles the payment lifecycle for a customer order. It validates card details, calls the payment gateway API, and updates the order status in the database."

Far more useful. The model now behaves like a **helpful assistant**.

Instruction-Tuned: When to Use



Chatbots and virtual assistants



Question-answering systems



Content-generation tools

3. RLHF Models

Reinforcement Learning from Human Feedback



Goes a step beyond instruction tuning



Human evaluators rank model outputs



Those rankings train a reward model



The LLM is then fine-tuned with reinforcement learning to maximise that reward



Result: responses aligned with human values and quality expectations

Examples: ChatGPT (GPT-4), Claude 3, Gemini

RLHF: How It Works

01

Collect human preference rankings on model outputs

02

Train a Reward Model (RM) on those rankings

03

Fine-tune the LLM using PPO (Proximal Policy Optimization) to maximise reward

Result: a model that is more helpful, harmless, and honest

RLHF: TechCorp's Experience

A frustrated customer writes:

> "YOUR APP DELETED ALL MY DATA. I WANT A REFUND NOW."

The RLHF-tuned model responds:

Category

URGENT - Data Loss

Suggested Response

"We sincerely apologize for this experience. Your data recovery has been flagged as high priority. Our team will contact you within 2 hours."

Calm, structured, and appropriate, because it was trained on **human preferences** for quality interactions.

RLHF: When to Use



Customer-facing applications



Situations requiring safety and alignment



When output quality and tone consistency matter

4. Domain-Specific Fine-Tuned Models

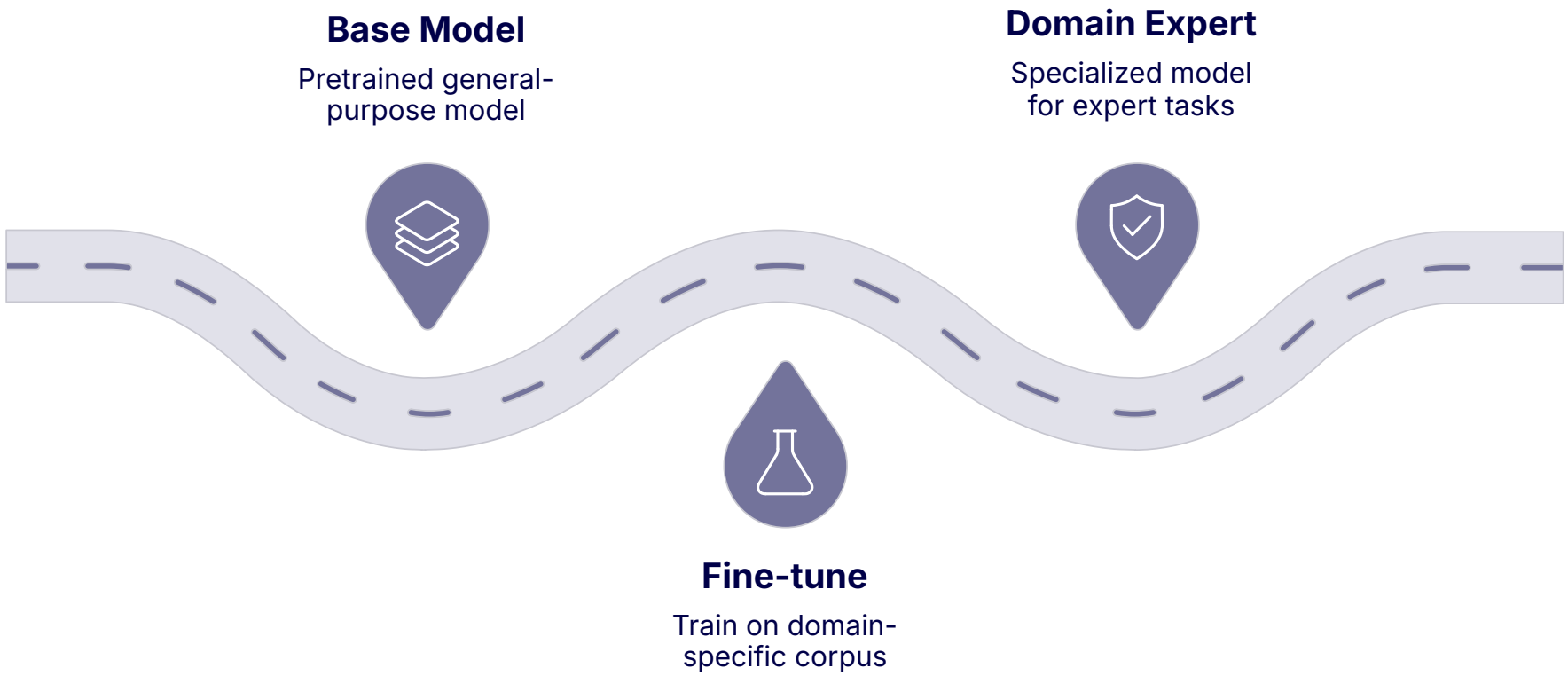
Take a pretrained or instruction-tuned model

Continue training on a curated dataset from one domain: legal, medical, finance, or code

Examples: CodeLlama, BioMedLM, FinGPT, StarCoder



Domain Fine-Tuning: How It Works



Example domain corpora:

Legal
contracts, case law, regulatory filings

Medical
clinical notes, research papers, drug databases

Code
GitHub repositories, Stack Overflow, documentation

Domain-Specific: TechCorp's Task

TechCorp needs to auto-generate boilerplate code. They compare a general instruction model with CodeLlama.

Prompt:

> "Write a Python function to connect to a PostgreSQL database using environment variables."

Output: General Instruction Model

```
import os
import psycopg2

def connect():
    conn = psycopg2.connect(os.getenv("DB_URL"))
    return conn
```

Works, but minimal: no error handling, no docs.

Output: CodeLlama

```
import os
import psycopg2
from psycopg2 import OperationalError

def connect_to_database():
 """
 Establishes a connection to a PostgreSQL database using environment
 variables for configuration.
 """
 try:
 conn = psycopg2.connect(
 host=os.getenv("DB_HOST", "localhost"),
 port=int(os.getenv("DB_PORT", 5432)),
 database=os.getenv("DB_NAME"),
 user=os.getenv("DB_USER"),
 password=os.getenv("DB_PASSWORD")
 )
 return conn
 except OperationalError as e:
 print(f"Database connection failed: {e}")
 raise
```

Domain-Specific: The Verdict

CodeLlama produces code closer to **production quality**: error handling, a docstring, and configurable settings.

For code generation, the domain-specific model clearly wins.

When to use:



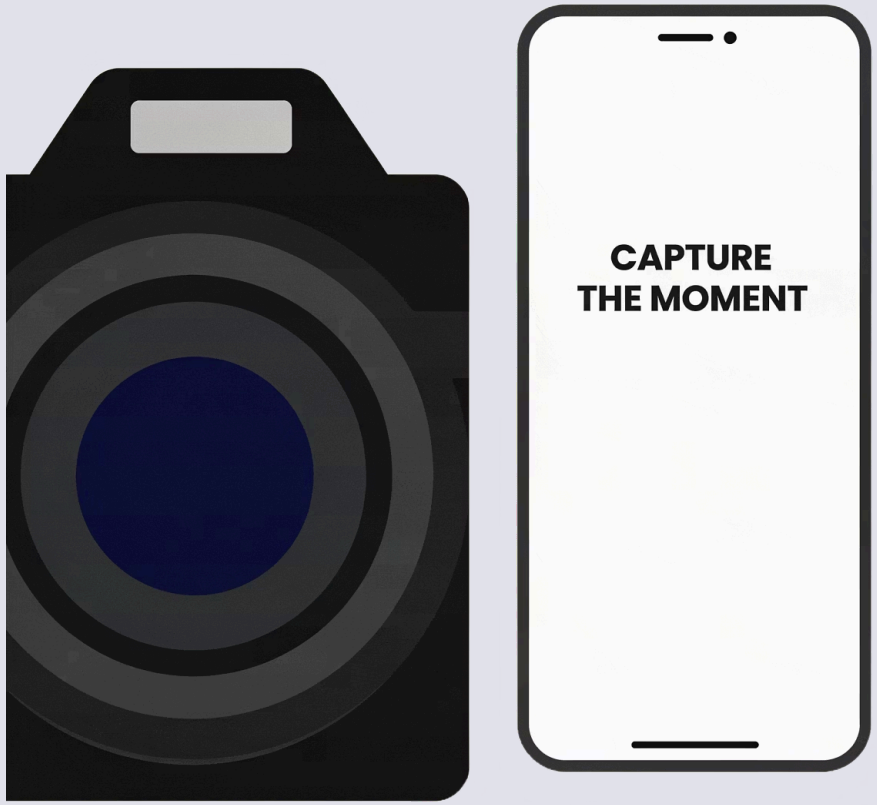
Specialised professional tasks (law, medicine, finance)



Code generation and review



Technical documentation

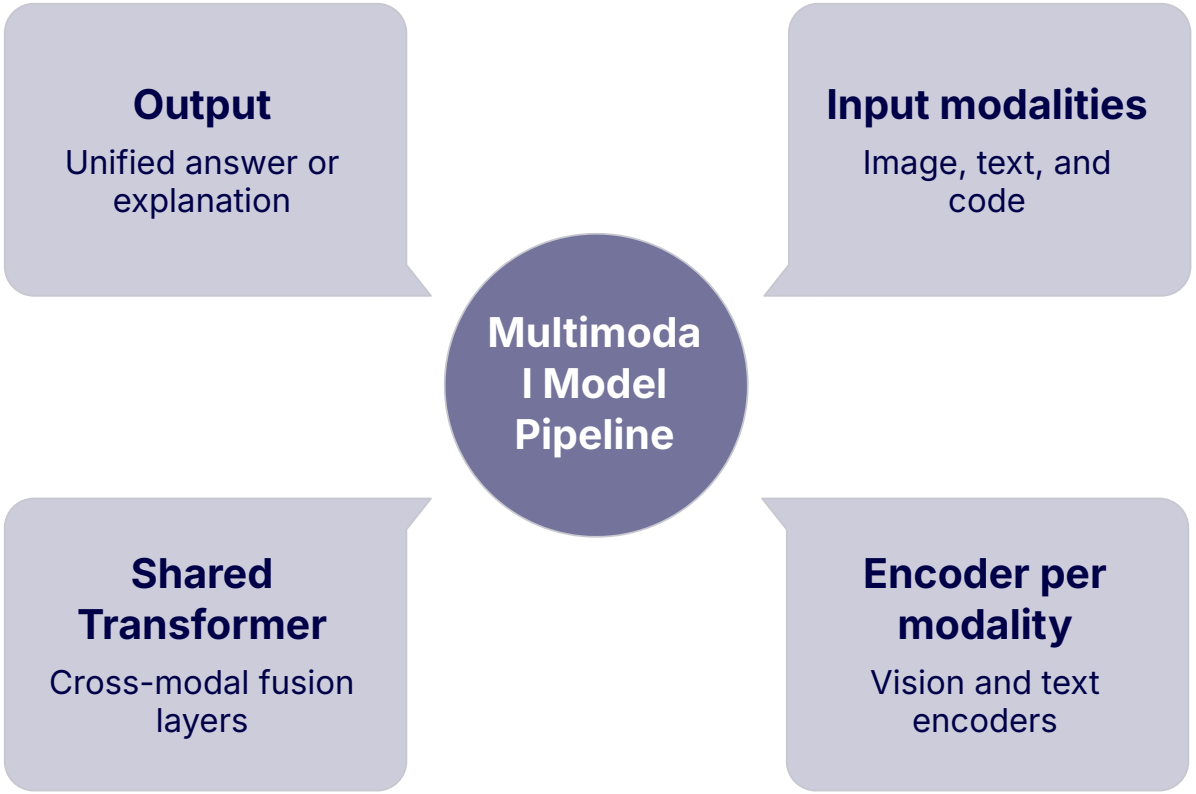


5. Multimodal Models

Process and generate content across multiple input types: text + images, or text + audio

Examples: GPT-4o, Gemini 1.5 Pro, Claude 3 Opus (images), LLaVA

Multimodal: How It Works



Example: **[screenshot of an error] + "Why is this failing?"**

1. Vision encoder + text encoder
2. Shared transformer layers
3. Answer: *"The error occurs because the variable conn is None. The database connection on line 42 failed silently."*

Multimodal: TechCorp's Experience

A developer pastes a **screenshot of a runtime error** and asks: "What is wrong here?"

The model reads the screenshot, identifies the stack trace, and explains the root cause, **without the developer typing the error out**.

When to use:



Applications involving images, charts, or diagrams



Document processing with mixed content



Accessibility tools



6. Small Language Models (SLMs)

Compact models, typically 1B to 7B parameters

Built to run on edge devices or lower compute budgets

Still strong on focused tasks

Examples: Phi-3 Mini, Gemma 2B, TinyLlama, Mistral 7B

SLMs: How They Are Built

High-quality curated data

(not just large data)

Knowledge distillation

from larger models

Quantization

to reduce memory footprint

FP32 weight (32 bits)

INT4 weight (4 bits)

Result: 8x smaller model, minimal accuracy loss for many tasks

SLMs: TechCorp's Experience

Goal: an IDE assistant that runs **locally on developer laptops, offline**.



A 70B-parameter model is too large



They deploy Phi-3 Mini (3.8B parameters)



Fits in about 4 GB of RAM



Responds in under one second

SLMs: When to Use



On-device or edge deployment



Cost-constrained environments



Latency-sensitive applications



Offline use cases

Summary: The Six Types

Model type	Training approach	Best for
Base	Next-token prediction on raw text	Research, fine-tuning starting point
Instruction-tuned	Fine-tuned on (instruction, response) pairs	Chatbots, Q&A, general assistants
RLHF	Reinforcement learning from human preferences	Aligned, safe, high-quality responses
Domain fine-tuned	Continued training on a specialised corpus	Code, legal, medical, financial tasks
Multimodal	Multiple encoders plus a shared transformer	Image + text tasks
Small Language Model	Distillation, quantization, quality data	Edge deployment, low latency

TechCorp's Final Choices

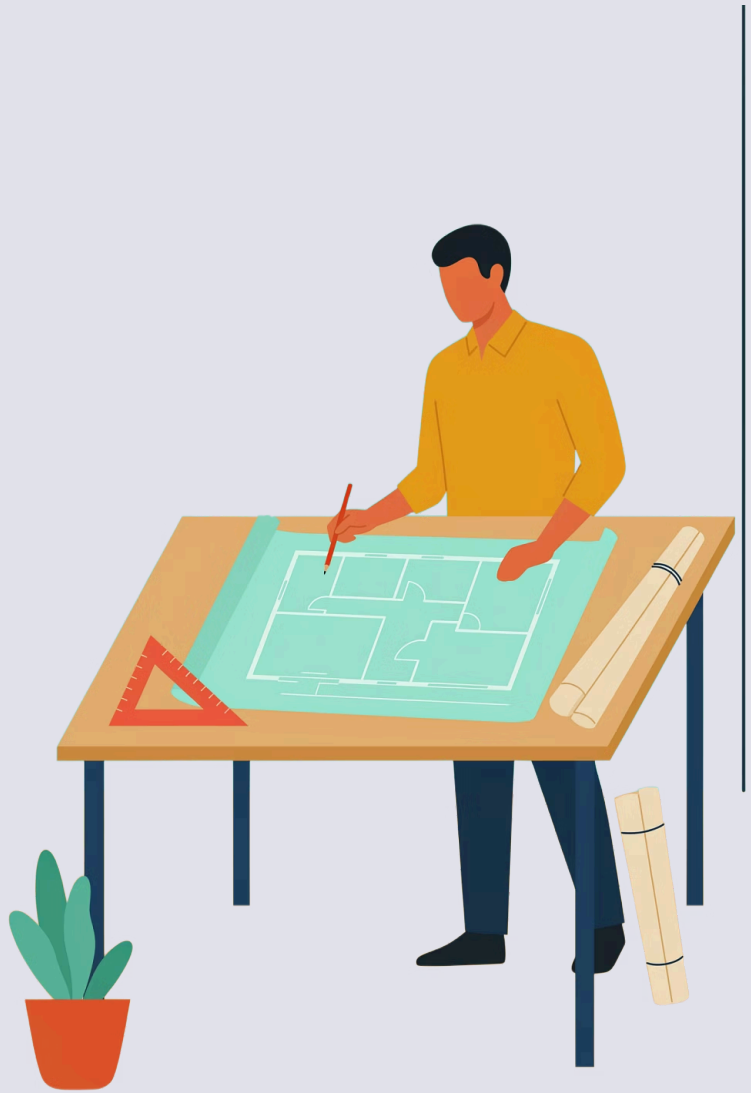
Task	Model type
Answer questions about the codebase	Instruction-tuned
Classify and respond to support tickets	RLHF
Generate boilerplate code	Domain fine-tuned (code)
Debug from screenshots	Multimodal
Local IDE assistant	Small Language Model
Using a base model directly	Not suitable

Key Takeaways

1. There is no single best LLM. The right model depends on task, deployment environment, and budget.
2. Base models are powerful starting points but need further tuning to be useful in products.
3. RLHF alignment is critical for user-facing apps where tone, safety, and consistency matter.
4. Domain fine-tuning can dramatically outperform general models on specialised tasks.
5. Multimodal models unlock new interactions but cost more compute.
6. Small models have closed the gap significantly and are often the right engineering choice when constraints exist.

Think Like an Architect

Understanding these distinctions lets engineers make **informed architectural decisions**, rather than defaulting to the largest or most popular model.



Quick Check: Discuss in Pairs

1

Why does a base model "continue" a question instead of answering it?

2

What problem does RLHF solve that instruction tuning alone does not?

3

You are building a medical-notes summariser for a hospital that cannot send data outside its network. Which types would you consider, and why?

4

When is a 3.8B model a better choice than a 70B model?